
LearnFlow

Architecture Decision Records

ADR-001 bis ADR-007

Version 1.0
Datum 2026-05-27
Autor Christoph A. Amstutz, MD-PhD
Status Alle ADRs: **Proposed**
Basis *LearnFlow_Requirements_v1.md*

Dieses Dokument enthält die Architecture Decision Records (ADRs) für das MVP der LearnFlow-Lernplattform. ADRs dokumentieren signifikante Architekturentscheidungen, die betrachteten Alternativen sowie die Begründung für den gewählten Ansatz. Alle ADRs haben den Status **Proposed** und sind zur Review freigegeben.

Inhaltsverzeichnis

1	Konfidenz-Scoring: Strategy Pattern mit austauschbarem Scorer	4
1.1	Kontext	4
1.2	Entscheidung	4
1.2.1	Komponentenstruktur	4
1.2.2	Interface-Definition	5
1.2.3	Stufenlogik (ConfidenceEvaluator)	5
1.2.4	MVP-Scorer: Semantische Ähnlichkeit	5
1.3	Begründung	6
1.4	Betrachtete Alternativen	6
1.4.1	Alternative 1 – Pure Function	6
1.4.2	Alternative 2 – Chain of Responsibility	6
1.5	Konsequenzen	6
1.6	Abhängigkeiten	6
1.7	Offen / Nächste Schritte	7
2	Fail-Safe RAG-Pipeline: Discriminated Union als Ergebnistyp	8
2.1	Kontext	8
2.2	Entscheidung	8
2.2.1	Ergebnistyp	8
2.2.2	Pipeline-Implementierung	9
2.2.3	Exhaustiveness im UI-Controller	9
2.2.4	LLM-Adapter-Interface mit Result-Typ	10
2.2.5	Flussdiagramm	10
2.3	Begründung	10
2.4	Betrachtete Alternativen	10
2.4.1	Alternative 1 – Exception-Hierarchie	11
2.4.2	Alternative 2 – Result-Monad (fp-ts / neverthrow)	11
2.4.3	Alternative 3 – Response-Objekt mit Status-Feld	11
2.5	Konsequenzen	11
2.6	Abhängigkeiten	11
2.7	Offen / Nächste Schritte	12
3	Konfigurationsservice: PostgreSQL mit In-Process-Cache und Audit-Log	13
3.1	Kontext	13
3.2	Entscheidung	13

3.2.1	Datenbankschema	13
3.2.2	Interface	14
3.2.3	Produktions-Implementierung (DBConfigService)	14
3.2.4	Zeitachse einer Schwellenwert-Änderung	15
3.3	Begründung	15
3.4	Betrachtete Alternativen	15
3.4.1	Alternative 1 – ENV-Variablen / Config-File	15
3.4.2	Alternative 2 – Redis mit Pub/Sub-Invalidierung	15
3.4.3	Alternative 3 – Polling ohne Cache	16
3.5	Konsequenzen	16
3.6	Abhängigkeiten	16
3.7	Offen / Nächste Schritte	16
4	LLM-Provider-Wahl: Azure OpenAI Service oder Anthropic Claude API	17
4.1	Kontext	17
4.1.1	Bewertungskriterien	17
4.2	Entscheidung	17
4.2.1	Pfad A – Azure OpenAI Service (<i>wenn EU/EWR-Datenresidenz gefordert</i>)	17
4.2.2	Pfad B – Anthropic Claude API (<i>wenn US-Datenresidenz akzeptiert</i>) . . .	18
4.2.3	Gemeinsame Konfiguration (beide Pfade)	18
4.2.4	Kostenabschätzung MVP	18
4.3	Begründung	18
4.4	Betrachtete Alternativen	19
4.4.1	Alternative 1 – OpenAI API direkt	19
4.4.2	Alternative 2 – Self-Hosted (Ollama + Llama 3.1 70B)	19
4.4.3	Alternative 3 – Mistral AI (EU-Provider)	19
4.5	Konsequenzen	19
4.6	Abhängigkeiten	19
4.7	Offen / Nächste Schritte	19
5	RAG-Stack: pgvector und multilinguales Embedding-Modell	21
5.1	Kontext	21
5.2	Entscheidung	21
5.2.1	Komponente 1 – Vektor-Datenbank: pgvector	21
5.2.2	Komponente 2 – Embedding-Modell: multilingual-e5-large-instruct	22
5.2.3	Komponente 3 – Chunking-Strategie	22
5.2.4	Gesamt-Architektur	22
5.2.5	Retriever-Interface	23
5.3	Begründung	24
5.4	Betrachtete Alternativen	24
5.4.1	Alternative 1 – Qdrant (dediziert, self-hosted)	24

5.4.2	Alternative 2 – Chroma (lokal)	24
5.4.3	Alternative 3 – OpenAI text-embedding-3-small (API)	24
5.5	Konsequenzen	24
5.6	Abhängigkeiten	25
5.7	Offen / Nächste Schritte	25
6	Authentifizierungs-Strategie: JWT mit HTTP-only Cookie und SSO-Ready-Middleware	26
6.1	Kontext	26
6.2	Entscheidung	26
6.2.1	Token-Strategie	26
6.2.2	Request-Flow	27
6.2.3	AuthMiddleware-Interface	27
6.2.4	RBAC-Deklaration pro Route	27
6.2.5	Passwort-Hashing (Argon2id)	28
6.2.6	Datenbankschema	28
6.3	Begründung	28
6.4	Betrachtete Alternativen	29
6.4.1	Alternative 1 – Auth.js (NextAuth.js)	29
6.4.2	Alternative 2 – Server-Side Session (express-session)	29
6.4.3	Alternative 3 – Keycloak (Self-Hosted IdP)	29
6.5	Konsequenzen	29
6.6	Abhängigkeiten	29
6.7	Offen / Nächste Schritte	30
7	Frontend-Framework: Next.js mit TypeScript und App Router	31
7.1	Kontext	31
7.2	Entscheidung	31
7.2.1	Architektur-Übersicht	31
7.2.2	TypeScript als Pflicht (Verbindung zu ADR-002)	32
7.2.3	LLM-Streaming im Chat-Interface (App Router)	32
7.2.4	Auth.js-Integration für Post-MVP SSO	33
7.2.5	Verzeichnisstruktur	33
7.3	Begründung	33
7.4	Betrachtete Alternativen	34
7.4.1	Alternative 1 – SvelteKit + TypeScript	34
7.4.2	Alternative 2 – React SPA (Vite) + Express.js	34
7.4.3	Alternative 3 – Vue 3 + Nuxt	34
7.5	Konsequenzen	34
7.6	Abhängigkeiten	34
7.7	Offen / Nächste Schritte	34

Konfidenz-Scoring: Strategy Pattern mit austauschbarem Scorer

Status	Proposed
Datum	2026-05-27
Autor	Christoph A. Amstutz, MD-PhD
Bezug	US-02, QA-06 (Testability), QA-04 (Reliability)
Massnahme	M2

1.1 Kontext

US-02 schreibt eine dreistufige Unterdrückungslogik für KI-Antworten vor. Der Entscheid vom 2026-05-20 legt die Vorrang-Reihenfolge fest:

1. **Quellenprüfung** — kein Retrieval-Treffer → Antwort wird nicht angezeigt
2. **Konfidenz-Score** — Score unter Schwellenwert → „Ich weiss es nicht“
3. **Self-Check-Anteil** — unter 80 % → „Eingeschränkt belegt“; unter 50 % → unterdrückt

Risiko 2 aus dem Requirements-Dokument benennt das zentrale offene Problem: *wie* der Self-Check-Anteil konkret berechnet wird, ist nicht definiert. Die zwei diskutierten Mechanismen — semantische Ähnlichkeit zwischen Antwort-Sätzen und Retrieval-Chunks vs. LLM-Selbstevaluation — haben unterschiedliche Qualitäts- und Kostenprofile. US-11 fordert zusätzlich, dass Schwellenwerte ohne Code-Deployment änderbar sind.

1.2 Entscheidung

Der Konfidenz-Scoring-Mechanismus wird als **Strategy Pattern** implementiert:

- Ein `ConfidenceScorer`-Interface kapselt ausschliesslich die Berechnung des Self-Check-Anteils.
- Ein `ConfidenceEvaluator` enthält die unveränderliche Stufenlogik und nimmt `ConfidenceScorer` sowie `ConfigService` als injizierte Abhängigkeiten entgegen.
- Schwellenwerte werden ausnahmslos über den `ConfigService` gelesen — nie als Konstanten im Code.

1.2.1 Komponentenstruktur

```
ConfidenceEvaluator
+-- ConfidenceScorer (Interface)
|   +-- SemanticSimilarityScorer    [MVP]
|   +-- LLMSelfEvaluationScorer     [Post-MVP]
|   '-- FixedRatioScorer           [Tests]
'-- ConfigService (Interface)
    +-- DBConfigService             [Produktion]
    '-- InMemoryConfigService       [Tests]
```

1.2.2 Interface-Definition

```

1 interface ConfidenceScorer {
2   /** Gibt einen Wert zwischen 0.0 und 1.0 zurueck:
3    * Anteil der Antwort-Aussagen, die durch Retrieval-Chunks
4     * belegbar sind. */
5   computeSelfCheckRatio(
6     llmResponse: string,
7     retrievalChunks: Chunk[]
8   ): Promise<number>
9 }

```

1.2.3 Stufenlogik (ConfidenceEvaluator)

```

1 type AnswerDecision =
2   | { kind: 'NO_SOURCE' }
3   | { kind: 'DONT_KNOW'; hint: string }
4   | { kind: 'SUPPRESSED' }
5   | { kind: 'LIMITED' }
6   | { kind: 'OK' }
7
8 class ConfidenceEvaluator {
9   constructor(
10     private scorer: ConfidenceScorer,
11     private config: ConfigService
12   ) {}
13
14   async evaluate(hits: Chunk[], score: number, response: string)
15     : Promise<AnswerDecision> {
16     const t = await this.config.getThresholds()
17
18     if (hits.length === 0)
19       return { kind: 'NO_SOURCE' } // Stufe 1
20
21     if (score < t.minConfidenceScore)
22       return { kind: 'DONT_KNOW',
23         hint: 'Versuche, einen konkreten Prozess zu nennen.' } // Stufe 2
24
25     const ratio = await this.scorer.computeSelfCheckRatio(response,
26       hits)
27     if (ratio < t.suppressThreshold) return { kind: 'SUPPRESSED' } // Stufe 3a
28     if (ratio < t.limitedThreshold) return { kind: 'LIMITED' } // Stufe 3b
29     return { kind: 'OK' }
30   }
31 }

```

1.2.4 MVP-Scorer: Semantische Ähnlichkeit

Für das MVP wird `SemanticSimilarityScorer` implementiert: jeder Satz der LLM-Antwort wird vektorisiert und mit der Cosinus-Ähnlichkeit zu den Retrieval-Chunks verglichen. Überschreitet ein Satz den Ähnlichkeitsschwellenwert, gilt er als „belegt“. Der Self-Check-Anteil ist der Quotient belegter Sätze zu Gesamtsätzen.

1.3 Begründung

Warum Strategy Pattern statt Pure Function? Eine Pure Function `evaluateConfidence(input): AnswerDecision` wäre ausreichend, wenn der Self-Check-Algorithmus feststünde. Da Risiko 2 explizit offen ist und sich der Algorithmus nach dem Pilot voraussichtlich ändern wird, würde eine Pure Function einen späteren Umbau erzwingen, der die gesamte Stufenlogik tangiert. Das Strategy Pattern trennt die stabile Stufenlogik vom instabilen Algorithmus. Kosten: eine Abstraktion und ca. 30 Minuten Mehraufwand.

Warum kein Chain-of-Responsibility? Drei fixe, geordnete Stufen rechtfertigen kein Pipeline-Pattern. Der Mehraufwand für ein Context-Objekt und Handler-Verkettung übersteigt den Nutzen für einen Anwendungsfall dieser Grösse.

1.4 Betrachtete Alternativen

1.4.1 Alternative 1 – Pure Function

```
1 function evaluateConfidence(input: ScoringInput): AnswerDecision {
    ... }
```

Abgelehnt weil der Self-Check-Algorithmus (Parameter `selfCheckRatio`) von aussen übergeben werden müsste — die Berechnung läge dann ungekapselt im aufrufenden Code. Das verlagert das Problem, löst es nicht.

1.4.2 Alternative 2 – Chain of Responsibility

Jede Stufe ist ein eigenständiger Handler; ein Context-Objekt wandert durch die Kette.

Abgelehnt Drei fixe, sequenzielle Stufen mit eindeutiger Reihenfolge sind kein Anwendungsfall für dieses Pattern. Erhöhter Boilerplate ohne Mehrwert.

1.5 Konsequenzen

Positiv

- Self-Check-Algorithmus austauschbar ohne Änderung an der Stufenlogik
- `ConfidenceEvaluator` mit `FixedRatioScorer` und `InMemoryConfigService` in unter 5 Unit-Tests vollständig abgedeckt
- Schwellenwerte nie als Magic Numbers im Code — alle Änderungen laufen über `ConfigService` (→ ADR-003)
- Post-MVP ML-basierter Scorer als neue Implementierung einhängbar

Negativ / Risiken

- Zwei Interfaces statt einer Funktion: minimaler Erklärungsaufwand beim Onboarding
- `SemanticSimilarityScorer` benötigt Zugriff auf den Embedding-Adapter (→ Abhängigkeit auf LLM-Infrastruktur auch im Scoring-Pfad)
- Qualität des MVP-Scorers erst durch RAG-Evaluierungs-Harness validierbar — kein theoretischer Nachweis möglich

1.6 Abhängigkeiten

Abhängigkeit	Typ	Hinweis
ADR-003 (ConfigService)	Voraussetzung	Schwellenwerte kommen aus ConfigService
ADR-002 (RAG-Pipeline)	Verwendend	ConfidenceEvaluator wird in der Pipeline aufgerufen
QA-06 Testability (M3)	Validierung	RAG-Evaluierungs-Harness prüft die 90 %-„Weiss-ich-nicht“-Rate
Risiko 2	Offen	Self-Check-Algorithmus muss vor Sprint 1 als Spike validiert werden

1.7 Offen / Nächste Schritte

- ☐ Spike: SemanticSimilarityScorer gegen Evaluierungs-Dataset ausführen; Qualitätsschwelle festlegen (Go/No-Go für Risiko 1+2)
- ☐ Konkrete Ähnlichkeitsschwelle für „Satz gilt als belegt“ definieren (Cosinus-Score $> X$)
- ☐ Entscheiden, ob selfCheckRatio im Response-Objekt für Lara sichtbar ist oder nur als interne Metainfo bleibt

Fail-Safe RAG-Pipeline: Discriminated Union als Ergebnistyp

Status	Proposed
Datum	2026-05-27
Autor	Christoph A. Amstutz, MD-PhD
Bezug	US-01, US-02, QA-04 (Reliability), QA-06 (Testability)
Massnahme	M5

2.1 Kontext

Die RAG-Pipeline ist der kritische Pfad von LearnFlow:

```
Eingabe-Validierung -> Retrieval -> Konfidenz-Check
                        -> LLM-Call -> Self-Check -> Antwort-Assembly
```

An jedem Schritt kann ein anderer Fehler auftreten, der eine andere Reaktion erfordert:

Fehlerzustand	Korrekte Reaktion	Falsche Reaktion
Kein Retrieval-Treffer	„Keine Quelle“ anzeigen	Antwort ohne Quelle generieren
Konfidenz zu tief	„Ich weiss es nicht“ + Hinweis	Antwort mit tiefer Konfidenz ausgeben
LLM-API nicht erreichbar	Klare Fehlermeldung	Leere oder halluzinierte Antwort
LLM-Timeout	Fehlermeldung nach 1 Retry	Unbegrenztes Warten
Self-Check < 50 %	Antwort vollständig unterdrücken	Unmarkierte Antwort ausgeben

Das Requirements-Dokument formuliert: „Eine falsch dargestellte Antwort ist schlimmer als gar keine.“ Das Kernproblem mit Exception-Hierarchien: ein vergessener `catch`-Branch gibt implizit `undefined` oder einen Teilzustand zurück — der exakte Mechanismus, der halluzinierte Antworten bei Ausfällen ermöglicht.

2.2 Entscheidung

Die RAG-Pipeline gibt **immer** einen explizit typisierten `RAGResult`-Wert zurück. Der Rückgabetypp ist ein **Discriminated Union**: eine abgeschlossene Menge benannter Varianten, die alle möglichen Pipeline-Ausgaben beschreiben.

2.2.1 Ergebnistyp

```
1 type RAGResult =
2   | { kind: 'answer'; content: string; sources: Source[];
3     confidence: number; label: 'ok' | 'limited' }
4   | { kind: 'dont_know'; hint: string }
5   | { kind: 'no_source' }
6   | { kind: 'suppressed' }
7   | { kind: 'unavailable'; message: string }
```

```
8 | { kind: 'timeout' }
```

Jede Variante entspricht direkt einem UI-Zustand. Es gibt keine Variante, die eine halluzinierte oder ungeprüfte Antwort transportiert.

2.2.2 Pipeline-Implementierung

```
1 async function runRAGPipeline(question: string): Promise<RAGResult>
2 {
3   const hits = await retriever.search(question) // wirft nie -- []
4   // = kein Treffer
5
6   const score = computeRetrievalScore(hits)
7   const decision = await evaluator.evaluate(hits, score, '')
8   if (decision.kind === 'NO_SOURCE') return { kind: 'no_source' }
9   if (decision.kind === 'DONT_KNOW') return { kind: 'dont_know',
10     hint: decision.hint }
11
12   const llmResult = await llmAdapter.complete(buildPrompt(question,
13     hits))
14   if (!llmResult.ok)
15     return llmResult.error.kind === 'timeout'
16       ? { kind: 'timeout' }
17       : { kind: 'unavailable', message: 'KI-Service nicht
18         erreichbar.' }
19
20   const final = await evaluator.evaluate(hits, score,
21     llmResult.value.text)
22   switch (final.kind) {
23     case 'SUPPRESSED': return { kind: 'suppressed' }
24     case 'LIMITED': return { kind: 'answer', content:
25       llmResult.value.text,
26       sources: extractSources(hits),
27       confidence: score, label: 'limited' }
28     case 'OK': return { kind: 'answer', content:
29       llmResult.value.text,
30       sources: extractSources(hits),
31       confidence: score, label: 'ok' }
32     default: return { kind: 'unavailable', message:
33       'Unerwarteter Zustand.' }
34   }
35 }
```

2.2.3 Exhaustiveness im UI-Controller

```
1 // Fehlender Case => TypeScript-Compilerfehler -- kein
2 // Runtime-Problem
3 function renderResult(result: RAGResult): ReactNode {
4   switch (result.kind) {
5     case 'answer': return result.label === 'limited'
6       ? <LimitedAnswerCard {...result} />
7       : <AnswerCard {...result} />
8     case 'dont_know': return <DontKnowCard hint={result.hint} />
9     case 'no_source': return <NoSourceCard />
10    case 'suppressed': return <NoSourceCard />
11    case 'unavailable': return <ErrorCard message={result.message}
12      />
13  }
```

```

11     case 'timeout':      return <ErrorCard
12         message="Zeitüberschreitung." />
13 }

```

2.2.4 LLM-Adapter-Interface mit Result-Typ

```

1 type LLMError =
2   | { kind: 'timeout'      }
3   | { kind: 'api_error'; statusCode: number; detail: string }
4   | { kind: 'rate_limit'  }
5
6 type Result<T, E> = { ok: true; value: T } | { ok: false; error: E }
7
8 interface LLMAdapter {
9   /** Gibt immer ein Result zurück -- wirft nie. */
10  complete(prompt: string): Promise<Result<LLMResponse, LLMError>>
11  stream(prompt: string): AsyncIterable<string>
12 }

```

2.2.5 Flussdiagramm

```

question
|
v
retriever.search()
+-(leer)-----> { kind: 'no_source' }
| hits
v
evaluator.evaluate() [pre-LLM]
+-(NO_SOURCE)-----> { kind: 'no_source' }
+-(DONT_KNOW)-----> { kind: 'dont_know', hint }
| pass
v
llmAdapter.complete()
+-(Err: timeout)-----> { kind: 'timeout' }
+-(Err: api_error / rate_limit)-----> { kind: 'unavailable' }
| Ok: llmResponse
v
evaluator.evaluate() [post-LLM, mit Self-Check]
+-(SUPPRESSED)-----> { kind: 'suppressed' }
+-(LIMITED)-----> { kind: 'answer', label:'limited' }
+-(OK)-----> { kind: 'answer', label:'ok' }

```

2.3 Begründung

Warum Discriminated Union statt Exception-Hierarchie? Alle sechs RAGResult-Varianten sind *erwartete* Zustände — direkt aus den Akzeptanzkriterien von US-01 und US-02 ableitbar. Der TypeScript-Compiler erzwingt den Exhaustiveness-Check im `switch`-Statement. Ein vergessener `catch`-Branch kann nicht zu einer halluzinierten Antwort führen.

Warum zwei `evaluator.evaluate()`-Aufrufe? Der erste Aufruf (pre-LLM) behandelt Stufe 1 und 2 — ohne LLM-Call prüfbar, spart API-Kosten bei frühen Exits. Der zweite Aufruf führt den Self-Check (Stufe 3) durch, der die LLM-Antwort benötigt.

2.4 Betrachtete Alternativen

2.4.1 Alternative 1 – Exception-Hierarchie

```
1 class RAGException extends Error {}
2 class NoSourceException extends RAGException {}
3 class LLMUnavailableException extends RAGException {}
```

Abgelehnt Kein Compiler-Exhaustiveness-Check. Vergessener `catch`-Branch führt zu undefiniertem Verhalten. Kontrollfluss via Exceptions ist schwerer nachzuvollziehen und zu testen.

2.4.2 Alternative 2 – Result-Monad (fp-ts / neverthrow)

Abgelehnt für MVP: Vollwertige Monad-Komposition erhöht die Einstiegshürde für das Team erheblich. Der Mehrwert gegenüber einfachen Discriminated Unions ist für diesen Anwendungsfall gering. Für ein Python-Backend ist der native `match`-Ausdruck (ab 3.10) die bessere Alternative.

2.4.3 Alternative 3 – Response-Objekt mit Status-Feld

```
1 type RAGResponse = { status: 'ok' | 'error' | 'dont_know'; data?:
  any; error?: any }
```

Abgelehnt Optionale Felder (?) entfernen die Compiler-Garantie. `result.data` könnte `undefined` sein, auch wenn `status === 'ok'`.

2.5 Konsequenzen

Positiv

- Vollständige Abdeckung aller Fehlerpfade ist vom Compiler erzwingbar
- Jede `RAGResult`-Variante isoliert und ohne Netzwerk testbar
- UI-Komponenten direkt auf Varianten gemappt — keine implizite Null-Prüfung nötig
- Neue Fehlerzustände können als neue Variante ergänzt werden; der Compiler zeigt sofort alle unbehandelten Stellen

Negativ / Risiken

- In Python nicht nativ (Klassen-Hierarchie oder `dataclass`-Varianten nötig); Exhaustiveness-Check muss manuell diszipliniert werden
- Die `Result<T, E>`-Abstraktion am LLM-Adapter ist für Entwickler ungewohnt, die ausschließlich exception-basiert gearbeitet haben
- Jede neue `RAGResult`-Variante erfordert Anpassung im UI-Controller

2.6 Abhängigkeiten

Abhängigkeit	Typ	Hinweis
ADR-001 (ConfidenceEvaluator)	Verwendet	Pipeline ruft <code>evaluator.evaluate()</code> zweimal auf
ADR-003 (ConfigService)	Indirekt	Über ADR-001 — Schwellenwerte aus Config
US-01 AC: Fehlermeldung bei Ausfall	Validierung	<code>unavailable</code> -Variante deckt dieses Kriterium ab
US-02 AC: Unterdrückung < 50 %	Validierung	<code>suppressed</code> -Variante deckt dieses Kriterium ab
QA-04 Circuit Breaker (M6)	Vorgelagert	Circuit Breaker gibt <code>Err(rate_limit)</code> zurück

2.7 Offen / Nächste Schritte

- ☐ Entscheiden: TypeScript oder Python für Backend? Bestimmt die Syntax des Discriminated Union
- ☐ Source-Typ definieren (Dokumenttitel, Abschnitt, Upload-Datum, Highlight-Offset für US-01)
- ☐ Timeout-Wert für LLM-Call konfigurierbar machen (→ ADR-003)
- ☐ Retry-Logik im LLM-Adapter spezifizieren: 1 Retry mit wie viel Backoff?

Konfigurationsservice: PostgreSQL mit In-Process-Cache und Audit-Log

Status	Proposed
Datum	2026-05-27
Autor	Christoph A. Amstutz, MD-PhD
Bezug	US-02, US-06, US-11, QA-04 (Reliability), QA-05 (Maintainability)
Massnahme	M7

3.1 Kontext

Drei User Stories setzen voraus, dass Systemparameter zur Laufzeit änderbar sind — ohne Code-Deployment und ohne Applikations-Neustart:

Parameter	Story	Standardwert
min_confidence_score	US-02	TBD (via Spike)
suppress_threshold	US-02	0.50
limited_threshold	US-02	0.80
stale_days	US-06	90

US-11 ergänzt: Konfigurationsänderungen werden mit Zeitstempel und ausführender Person protokolliert. Änderungen müssen sofort wirken. Das zentrale Problem mit ENV-Variablen oder Config-Files: sie erfordern einen Neustart.

3.2 Entscheidung

Konfigurationsparameter werden in der **bestehenden PostgreSQL-Datenbank** gespeichert. Ein zentraler ConfigService liest Werte über einen **In-Process-Cache mit TTL und sofortiger Invalidierung beim Schreiben**. Alle Schreiboperationen werden transaktionssicher in einem unveränderlichen **Audit-Log** protokolliert.

3.2.1 Datenbankschema

```
1 CREATE TABLE config_params (  
2     key          TEXT          PRIMARY KEY,  
3     value        TEXT          NOT NULL,  
4     updated_at   TIMESTAMPTZ  NOT NULL DEFAULT now()  
5 );  
6  
7 INSERT INTO config_params (key, value) VALUES  
8     ('min_confidence_score', '0.65'),  
9     ('suppress_threshold',   '0.50'),  
10    ('limited_threshold',     '0.80'),  
11    ('stale_days',           '90');  
12
```

```

13  -- Unveraenderliches Audit-Log (kein DELETE/UPDATE fuer die
    Applikation)
14  CREATE TABLE config_audit_log (
15      id          BIGSERIAL PRIMARY KEY,
16      key         TEXT      NOT NULL,
17      old_value   TEXT,      -- NULL bei Erstanlage
18      new_value   TEXT      NOT NULL,
19      changed_by  TEXT      NOT NULL, -- Username (kein FK)
20      changed_at  TIMESTAMPTZ NOT NULL DEFAULT now()
21  );

```

Tabelle	App-Rolle	Admin-UI-Rolle
config_params	SELECT, UPDATE	SELECT, UPDATE
config_audit_log	SELECT, INSERT	SELECT, INSERT

Kein DELETE, kein TRUNCATE auf config_audit_log — weder für die App noch für die Admin-UI.

3.2.2 Interface

```

1  interface ConfigService {
2      get(key: string): Promise<string>
3      getNumber(key: string): Promise<number>
4      getThresholds(): Promise<ConfidenceThresholds>
5      set(key: string, value: string, changedBy: string): Promise<void>
6  }
7
8  type ConfidenceThresholds = {
9      minConfidenceScore: number // 'min_confidence_score'
10     suppressThreshold: number  // 'suppress_threshold'
11     limitedThreshold: number   // 'limited_threshold'
12 }

```

3.2.3 Produktions-Implementierung (DBConfigService)

```

1  class DBConfigService implements ConfigService {
2      private cache = new Map<string, { value: string; expiresAt:
        number }>()
3      private readonly TTL_MS = 30_000 // 30 Sekunden
4
5      constructor(private db: Database) {}
6
7      async get(key: string): Promise<string> {
8          const cached = this.cache.get(key)
9          if (cached && Date.now() < cached.expiresAt) return cached.value
10
11         const row = await this.db.queryOne<{ value: string }>(<
12             'SELECT value FROM config_params WHERE key = $1', [key]
13         )
14         if (!row) throw new ConfigKeyNotFoundError(key)
15         this.cache.set(key, { value: row.value, expiresAt: Date.now() +
16             this.TTL_MS })
17         return row.value
18     }
19
20     async set(key: string, newValue: string, changedBy: string):
        Promise<void> {

```

```

20     const oldValue = await this.get(key).catch(() => null)
21     await this.db.transaction(async (tx) => {
22         await tx.execute(
23             'UPDATE config_params SET value = $1, updated_at = now()
24             WHERE key = $2',
25             [newValue, key])
26         await tx.execute(
27             'INSERT INTO config_audit_log (key, old_value, new_value,
28             changed_by) VALUES ($1,$2,$3,$4)',
29             [key, oldValue, newValue, changedBy])
30     })
31     this.cache.delete(key)    // sofortige Invalidation

```

3.2.4 Zeitachse einer Schwellenwert-Änderung

```

t=0s  Admin setzt suppress_threshold = 0.60 (via Admin-UI)
t=0s  ConfigService.set() -> DB UPDATE + Audit-Log INSERT (1 Transaktion)
t=0s  cache.delete('suppress_threshold')
t~1s  Nächste Anfrage: cache miss -> DB-Lesezugriff -> neuer Wert aktiv
      => Keine Neustart, kein Deployment erforderlich

[Single-Instance: Änderung wirkt innerhalb ~1 Sekunde]
[Multi-Instance (Post-MVP): bis zu 30s Inkonsistenz -> Redis-Upgrade]

```

3.3 Begründung

Warum nicht ENV-Variablen? ENV-Variablen werden beim Prozessstart eingelesen. Änderungen erfordern einen Neustart — widerspricht der expliziten MUST-Anforderung aus US-11.

Warum nicht Redis als primärer Config-Store? Im MVP ist eine einzige App-Instanz geplant (20 concurrent users). Redis als neue Infrastrukturkomponente für ein 30-Sekunden-Propagationsproblem, das im Pilotbetrieb nie auftreten wird, widerspricht dem Budget-Constraint. Der Upgrade-Pfad ist dokumentiert und ohne Interface-Änderung nachrüstbar.

Warum sofortige Cache-Invalidation statt reinem TTL? Rein TTL-basierter Cache bedeutet: direkt nach einer Admin-Änderung sieht dieselbe Instanz für bis zu 30 Sekunden den alten Wert. Sofortige Invalidation bei `set()` kostet eine Zeile Code und eliminiert das Problem vollständig für Single-Instance.

Warum kein FK von `changed_by` auf `users`? Administratoren könnten gelöscht werden (Mitarbeitendenwechsel). Ein FK würde den Löschvorgang blockieren oder das Audit-Log korrumpieren. Der Username als Text überlebt User-Löschoperationen.

3.4 Betrachtete Alternativen

3.4.1 Alternative 1 – ENV-Variablen / Config-File

Abgelehnt Änderungen erfordern Neustart. Verletzt MUST-Anforderung aus US-11 direkt.

3.4.2 Alternative 2 – Redis mit Pub/Sub-Invalidation

```

Admin -> UPDATE config_params (PostgreSQL)
      -> PUBLISH config:invalidate (Redis)

```



```
Alle Instanzen -> SUBSCRIBE -> cache.delete(key) [sofort]
```

Zurückgestellt Korrekte Architektur für Multi-Instance-Betrieb. Für MVP (Single-Instance) unverhältnismässig. Interface bleibt kompatibel — Upgrade ohne Business-Logik-Änderung möglich. **Upgrade-Trigger:** mehr als eine App-Instanz im Betrieb.

3.4.3 Alternative 3 – Polling ohne Cache

Abgelehnt Der ConfidenceEvaluator ruft `getThresholds()` bei jeder Nutzeranfrage auf. Bei 20 concurrent users entstehen 60–100 DB-Queries pro Sekunde allein für Config-Reads.

3.5 Konsequenzen

Positiv

- Keine neue Infrastrukturkomponente: Config in bestehender PostgreSQL-Instanz
- Audit-Log transaktionssicher: Änderung und Log gelingen oder scheitern gemeinsam
- InMemoryConfigService macht alle Komponenten ohne DB testbar
- Schwellenwerte nie als Magic Numbers im Code

Negativ / Risiken

- Bei zukünftigem Multi-Instance-Betrieb: bis zu 30 s Inkonsistenz (akzeptabel für MVP; bei Scale-Up adressieren)
- DB-Ausfall macht Config-Reads unmöglich: Fallback auf gecachte Werte (bis TTL abläuft) vorhanden, aber kein expliziter Kaltstart-Fallback definiert

3.6 Abhängigkeiten

Abhängigkeit	Typ	Hinweis
ADR-001 (ConfidenceEvaluator)	Konsument	Liest <code>getThresholds()</code> bei jeder Anfrage
ADR-002 (RAG-Pipeline)	Indirekt	Über ADR-001
US-11 Admin-Seite	Schreibweg	Admin-UI ruft <code>ConfigService.set()</code> auf
QA-03 Security	Voraussetzung	Admin-Seite nur für Admin-Rolle erreichbar
PostgreSQL	Infrastruktur	Kein zusätzliches System erforderlich

3.7 Offen / Nächste Schritte

- ☐ Initiale Schwellenwerte für `min_confidence_score` nach RAG-Spike festlegen und in Seed-Migration eintragen
- ☐ DB-Migrationsscript für `config_params` und `config_audit_log` erstellen (Teil von Sprint 0)
- ☐ Entscheiden: Admin-UI (US-11) zeigt Audit-Log als read-only Tabelle — Scope für MVP?
- ☐ Kaltstart-Verhalten bei leerem Cache und DB-Ausfall definieren (Default-Werte im Code als letzter Fallback?)

LLM-Provider-Wahl: Azure OpenAI Service oder Anthropic Claude API

Status	Proposed
Datum	2026-05-27
Autor	Christoph A. Amstutz, MD-PhD
Bezug	US-01, US-02, QA-03 (Security), QA-04 (Reliability), Risiko 3
Abhängigkeit	ADR-002 (LLMAdapter-Interface), ADR-005 (RAG-Stack)

4.1 Kontext

LearnFlow schickt interne Unternehmensdokumente als Kontext an ein LLM. Das stellt die Provider-Wahl unter zwei Zwänge, die über reine Qualitäts- und Kostenüberlegungen hinausgehen:

Datenschutz / Datenresidenz: Jeder Prompt enthält Ausschnitte aus vertraulichen internen Dokumenten. Je nach Unternehmensstandort (CH/DE/AT) kann die Übermittlung an einen US-amerikanischen API-Endpoint compliance-relevant sein.

API-Quota (Risiko 3): Quota-Anfragen können mehrere Wochen dauern. Die Provider-Entscheidung muss deshalb in Sprint 0 getroffen und die Quota-Anfrage sofort gestellt werden.

Das LLM-Adapter-Interface aus ADR-002 stellt sicher, dass der Provider später ohne Änderung an der Business-Logik ausgetauscht werden kann.

4.1.1 Bewertungskriterien

Kriterium	Gewicht	Begründung
Datenresidenz / DSGVO-Compliance	Hoch	Interne Dokumente im Prompt
RAG-Qualität (Long-Context, Präzision)	Hoch	Kernversprechen der Plattform
Quota-Verfügbarkeit für MVP	Hoch	Risiko 3
Kosten (Token-Preis $\times$ Volumen)	Mittel	480h Budget, ≈ 20 User
Streaming-Support	Mittel	$p_{95} \leq 10$ s (QA-01)
Embedding-Verfügbarkeit beim Provider	Mittel	Vereinfacht ADR-005

4.2 Entscheidung

Zweistufige Empfehlung abhängig von der Compliance-Anforderung:

4.2.1 Pfad A – Azure OpenAI Service (wenn EU/EWR-Datenresidenz gefordert)

Azure OpenAI bietet dieselben Modelle (GPT-4o, GPT-4o-mini) über Microsoft-Infrastruktur in europäischen Rechenzentren an und stellt einen EU-Datenverarbeitungsvertrag (DPA nach Art. 28 DSGVO) bereit.

Empfohlene Modelle:

- **GPT-4o-mini** — Hauptmodell: niedrige Latenz, günstiger Token-Preis
- **GPT-4o** — Fallback für komplexe, mehrstufige Kontextfragen

4.2.2 Pfad B – Anthropic Claude API (wenn US-Datenresidenz akzeptiert)

Claude 3.5 Haiku / Sonnet bieten hervorragende Qualität für Retrieval-gestützte Antwortgenerierung mit präziser Quellenreferenzierung.

Empfohlene Modelle:

- **Claude 3.5 Haiku** — Hauptmodell: $p_{50} \approx 2-3s$, günstiger als Sonnet
- **Claude 3.5 Sonnet** — Fallback für komplexe Fragen

Entscheidungsblockierer: Die Compliance-Frage (Datenresidenz) muss durch die IT-/Legal-Abteilung bis Ende Sprint 0 beantwortet werden. Bis dahin bleibt dieser ADR **Proposed**. Die technische Implementierung ist für beide Pfade identisch (LLM-Adapter-Interface, ADR-002).

4.2.3 Gemeinsame Konfiguration (beide Pfade)

```

1 interface LLMAdapter {
2     complete(prompt: string, options?: CompletionOptions)
3         : Promise<Result<LLMResponse, LLMError>>
4     stream(prompt: string, options?: CompletionOptions):
5         AsyncIterable<string>
6 }
7
8 type CompletionOptions = {
9     maxTokens?: number // Default: 1024
10    temperature?: number // Default: 0.1 (faktische Antworten)
11    timeoutMs?: number // aus ConfigService
12 }

```

Modellname und Timeout werden als Config-Parameter gespeichert (→ ADR-003): `llm_model`, `llm_max_tokens`, `llm_temperature`, `llm_timeout_ms`. Provider-Wechsel innerhalb desselben Anbieters erfordert nur eine DB-Änderung.

4.2.4 Kostenabschätzung MVP

	Azure GPT-4o-mini	Claude 3.5 Haiku
Input (pro 1M Token)	≈\$0.15	≈\$0.80
Output (pro 1M Token)	≈\$0.60	≈\$4.00
Kosten/Anfrage (≈3.5k Token)	≈\$0.001	≈\$0.005
Kosten bei 1.000 Anfragen/Monat	≈\$1.–	≈\$5.–

4.3 Begründung

Warum kein Self-Hosted-Modell? GPU-Infrastruktur liegt ausserhalb des 480h-Budgets. Qualität von Open-Source-Modellen bei deutschen Fachdomänen-Dokumenten ist noch nicht auf Cloud-Niveau. Der LLM-Adapter lässt diesen Wechsel Post-MVP zu, ohne Business-Logik zu ändern.

Warum nicht OpenAI direkt (ohne Azure)? OpenAI API bietet keine europäische Datenresidenz-Garantie. Wenn Compliance irrelevant ist, wäre OpenAI direkt technisch gleich-

wertig mit Pfad B (Anthropic) und kostengünstiger als Pfad A.

4.4 Betrachtete Alternativen

4.4.1 Alternative 1 – OpenAI API direkt

Zurückgestellt Kein EU-Datenresidenz-Angebot. Wenn Compliance keine Rolle spielt, technisch gleichwertig mit Pfad B und kostengünstiger als Pfad A.

4.4.2 Alternative 2 – Self-Hosted (Ollama + Llama 3.1 70B)

Abgelehnt GPU-Infrastruktur ausserhalb des Budgets. Qualität bei deutschen Fachdokumenten ungesichert. Betriebsaufwand für 1-FTE-Maintenance zu hoch.

4.4.3 Alternative 3 – Mistral AI (EU-Provider)

Zurückgestellt Europäische Datenresidenz vorhanden. Mistral-Modelle bei deutschen Domänentexten weniger evaluiert. Kleineres Ökosystem. Viable Post-MVP-Alternative.

4.5 Konsequenzen

Positiv

- LLM-Adapter-Interface hält Provider vollständig aus Business-Logik heraus
- Modell ohne Deployment wechselbar (ConfigService, ADR-003)
- Streaming in beiden Pfaden verfügbar — $p_{95} \leq 10$ s erreichbar
- Kostenvolumen bei Pilot-Nutzung minimal

Negativ / Risiken

- **Quota-Risiko** (Risiko 3): Azure-Quota-Anfragen können 2–4 Wochen dauern; Anthropic-Pläne schneller verfügbar. Quota-Anfrage sofort stellen.
- API-Key darf nie in Code, Logs oder DB erscheinen (→ QA-03, M10)
- Cloud-LLMs: kein garantiertes p_{95} -SLA; eigener Timeout und Circuit Breaker (ADR-002) als Absicherung nötig

4.6 Abhängigkeiten

Abhängigkeit	Typ	Hinweis
ADR-002 (LLMAdapter-Interface)	Vorausgesetzt	Provider hinter Interface gekapselt
ADR-003 (ConfigService)	Vorausgesetzt	Modellname, Timeout als Config-Parameter
ADR-005 (RAG-Stack)	Nachgelagert	Embedding-Provider-Wahl hängt von LLM-Provider ab
Risiko 3	Direkt	Quota-Anfrage blockiert ggf. Sprint-1-Start
Legal/IT-Abteilung	Entscheider	Compliance-Anforderung bis Ende Sprint 0

4.7 Offen / Nächste Schritte

- ☐ **Bis Ende Sprint 0:** Legal/IT bestätigt Datenresidenz-Anforderung → Pfad A (Azure) oder Pfad B (Anthropic)
- ☐ **Sofort nach Entscheid:** Quota-Anfrage beim gewählten Provider stellen
- ☐ Prompt-Template für RAG-Antworten mit Quellenangabe ausarbeiten und im Spike testen

- Rate-Limit-Werte des Providers dokumentieren → Circuit-Breaker-Konfiguration (ADR-002)

ADR-005

RAG-Stack: pgvector und multilinguales Embedding-Modell

Status	Proposed
Datum	2026-05-27
Autor	Christoph A. Amstutz, MD-PhD
Bezug	US-01, US-04, QA-06 (Testability), Risiko 1
Abhängigkeit	ADR-001 (ConfidenceScorer), ADR-002 (RAG-Pipeline), ADR-004 (LLM-Provider)

5.1 Kontext

Der RAG-Stack ist die technische Grundlage des Kernversprechens: verlässliche, quellenbelegte Antworten. Risiko 1 stuft die RAG-Qualität als existenzielles Go/No-Go-Risiko ein. Der Stack umfasst drei eng gekoppelte Entscheidungen: Vektor-Datenbank, Embedding-Modell und Chunking-Strategie.

Anforderung	Quelle	Implikation
Dokumente: PDF, DOCX, Markdown	US-04	Parser für 3 Formate nötig
≤50 Seiten → 5 min	US-04	Asynchrones Chunking + Embedding
Quellenreferenz: Titel + Abschnitt	US-01	Chunks müssen Herkunftsmetadaten tragen
Klick → Dokument mit Highlight	US-01	Chunks müssen Offset/Seitenzahl speichern
Sprache: Deutsch (MVP)	Req.	Embedding-Modell muss Deutsch unterstützen
Out-of-Corpus „Weiss nicht“ ≥ 90 %	US-02	Retrieval-Score als Konfidenz-Signal

5.2 Entscheidung

5.2.1 Komponente 1 – Vektor-Datenbank: pgvector

Für das MVP wird **pgvector** als Vektor-Datenbank eingesetzt — als Extension der bereits vorhandenen PostgreSQL-Instanz.

```
1 CREATE EXTENSION IF NOT EXISTS vector;
2
3 CREATE TABLE document_chunks (
4     id BIGSERIAL PRIMARY KEY,
5     document_id UUID NOT NULL REFERENCES documents(id) ON
      DELETE CASCADE,
6     chunk_index INTEGER NOT NULL,
7     content TEXT NOT NULL,
8     embedding vector(1024) NOT NULL,
9     page_number INTEGER,           -- fuer PDF-Highlight
      (US-01)
```

```

10 char_offset INTEGER,
11 token_count INTEGER NOT NULL,
12 created_at TIMESTAMPTZ NOT NULL DEFAULT now()
13 );
14
15 -- HNSW-Index fuer schnelle Aehnlichkeitssuche
16 CREATE INDEX ON document_chunks
17 USING hnsw (embedding vector_cosine_ops)
18 WITH (m = 16, ef_construction = 64);

1 -- Aehnlichkeitssuche mit Score und Bereichsisolation (US-04)
2 SELECT dc.content, dc.page_number, dc.char_offset,
3        d.title AS document_title, d.uploaded_at,
4        1 - (dc.embedding <=> $1::vector) AS cosine_similarity
5 FROM document_chunks dc
6 JOIN documents d ON d.id = dc.document_id
7 WHERE d.area_id = $2
8 ORDER BY dc.embedding <=> $1::vector
9 LIMIT $3;

```

5.2.2 Komponente 2 – Embedding-Modell: multilingual-e5-large-instruct

Eigenschaft	Wert
Modell	intfloat/multilingual-e5-large-instruct
Embedding-Dimension	1024
Max. Input-Token	512
Sprachen	100+, inkl. Deutsch
Modellgrösse	≈560 MB
CPU-Inferenz	≈150–300 ms / 512 Token
Lizenz	MIT

```

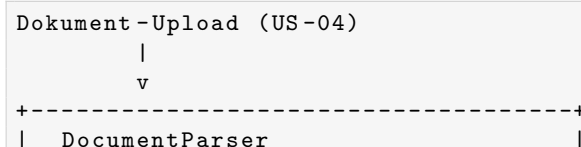
1 class EmbeddingAdapter:
2     def embed_query(self, query: str) -> list[float]:
3         # E5-Modelle: unterschiedliche Prefixes fuer Query vs.
4         # Passage
5         return self.model.encode(f"query: {query}").tolist()
6
7     def embed_document(self, text: str) -> list[float]:
8         return self.model.encode(f"passage: {text}").tolist()

```

5.2.3 Komponente 3 – Chunking-Strategie

Parameter	Wert
Strategie	Rekursiver Splitter (Hierarchie: \n\n → \n → Satz → Wort)
Chunk-Grösse	400 Token
Overlap	50 Token
Mindestgrösse	50 Token (filtert Artefakte)

5.2.4 Gesamt-Architektur



```

| PDF -> pdfplumber / pypdf |
| DOCX -> python-docx |
| MD -> direkt |
+-----+
| |
| v |
+-----+
| ChunkingService |
| Rekursiver Splitter |
| 400 Token / 50 Overlap |
+-----+
| |
| v |
+-----+
| EmbeddingAdapter |
| multilingual-e5-large-instruct |
+-----+
| |
| v |
+-----+
| VectorStore (pgvector) |
| INSERT document_chunks |
| HNSW-Index automatisch aktuell |
+-----+

Suchanfrage (US-01)
|
| v EmbeddingAdapter.embed_query()
| v
+-----+
| Retriever |
| SELECT ... ORDER BY cosine dist |
| WHERE area_id = {bereich} |
| LIMIT top_k (Default: 5) |
+-----+
| |
| v --> RAG-Pipeline (ADR-002)

```

5.2.5 Retriever-Interface

```

1 interface Retriever {
2     search(query: string, areaId: string, topK?: number):
        Promise<RetrievedChunk[]>
3 }
4
5 type RetrievedChunk = {
6     content: string
7     similarity: number // 0.0 -- 1.0
8     documentTitle: string
9     uploadedAt: Date
10    pageNumber: number | null
11    charOffset: number
12    documentId: string
13 }
14
15 class InMemoryRetriever implements Retriever {
16     constructor(private fixtures: RetrievedChunk[]) {}
17     async search(): Promise<RetrievedChunk[]> { return this.fixtures }
18 }

```


5.3 Begründung

Warum pgvector statt dedizierter Vektor-DB (Qdrant, Chroma)? pgvector nutzt die bereits vorhandene PostgreSQL-Instanz — keine neue Infrastruktur, kein zusätzliches Betriebswissen. Der HNSW-Index ist für 500 Dokumente und 20 concurrent users performant genug. Der Retriever-Interface-Layer erlaubt den Wechsel zu Qdrant transparent.

Warum lokales Embedding-Modell statt API? (1) Kein zweiter externer Service = keine zweite Quota-Abhängigkeit (Risiko 3). (2) Keine Embedding-Kosten beim Re-Indexieren. (3) Dokumenteninhalt verlässt das System nicht — konsistente Datenresidenz wenn ADR-004 auf Azure OpenAI zeigt.

Warum multilingual-e5-large und nicht kleiner? multilingual-e5-large übertrifft e5-small bei deutschen Domänentexten deutlich (MTEB-Benchmark). 560 MB Modellgrösse sind bei Server-Deployment kein Problem. multilingual-e5-base ist Fallback wenn CPU-Inferenzzeit zu hoch.

5.4 Betrachtete Alternativen

5.4.1 Alternative 1 – Qdrant (dediziert, self-hosted)

Zurückgestellt Für MVP-Corpus (500 Dokumente, 1 Bereich) Overkill. Neuer Infrastruktur-Container, Betriebsaufwand. Upgrade-Pfad via Interface.

5.4.2 Alternative 2 – Chroma (lokal)

Abgelehnt Kein SQL-Join möglich (Bereichsisolation). Transaktionssicherheit schlechter als PostgreSQL. Filesystem-Persistenz problematisch bei Backups.

5.4.3 Alternative 3 – OpenAI text-embedding-3-small (API)

Zurückgestellt Zweite externe API-Abhängigkeit (Risiko 3). Datenschutzinkonsistenz wenn LLM-Provider Azure ist, Embeddings aber zu OpenAI US gehen. Viable wenn lokale Compute-Infrastruktur Problem ist.

5.5 Konsequenzen

Positiv

- Keine neue Infrastrukturkomponente für MVP
- Bereichsisolation nativ in SQL (`WHERE area_id = ?`)
- Lokales Embedding: keine API-Kosten, keine externen Daten beim Upload
- `InMemoryRetriever` macht alle Tests ohne DB und Embedding-Modell ausführbar

Negativ / Risiken

- pgvector HNSW: bei > 1 M Chunks degradiert Performance. Mitigation: Upgrade auf Qdrant via Interface
- Lokales Modell: ≈ 560 MB auf dem Server; beim Start laden (kein Cold-Start pro Request)
- **Go/No-Go-Risiko (Risiko 1):** Retrieval-Qualität auf echten Dokumenten muss im Spike vor Sprint 1 validiert werden

Abhängigkeit	Typ	Hinweis
ADR-002 (RAG-Pipeline)	Konsument	<code>Retriever.search()</code> ist erster Pipeline-Schritt
ADR-001 (ConfidenceScorer)	Konsument	<code>SemanticSimilarityScorer</code> nutzt denselben <code>EmbeddingAdapter</code>
ADR-004 (LLM-Provider)	Koordiniert	Datenresidenz für Embeddings muss konsistent sein
ADR-003 (ConfigService)	Konsument	<code>top_k</code> , Ähnlichkeitsschwelle als Config-Parameter
Risiko 1	Go/No-Go	Spike mit Evaluierungs-Harness vor Sprint 1

5.6 Abhängigkeiten

5.7 Offen / Nächste Schritte

- ☐ **Sprint 0 Spike:** multilingual-e5-large mit echten Pilotdokumenten und Evaluierungs-Dataset testen; Source-Hit-Rate, Out-of-Corpus-Rate, p_{95} -Latenz messen
- ☐ Chunk-Grösse variieren (256 / 400 / 512 Token) und optimalen Wert messen
- ☐ PDF-Tabellenextraktion evaluieren: pdfplumber vs. pymupdf für Layoutqualität
- ☐ Re-Indexierung bei Dokumenten-Update (gleicher Dateiname, US-04): alte Chunks löschen, neue insertieren — in einer Transaktion
- ☐ `top_k` und Ähnlichkeitsschwellenwert in `config_params` eintragen (→ ADR-003)

Authentifizierungs-Strategie: JWT mit HTTP-only Cookie und SSO-Ready-Middleware

Status	Proposed
Datum	2026-05-27
Autor	Christoph A. Amstutz, MD-PhD
Bezug	US-05, QA-03 (Security), QA-05 (Maintainability)
Abhängigkeit	ADR-007 (Frontend-Framework)

6.1 Kontext

US-05 definiert die MVP-Authentifizierungsanforderungen:

- Username/Passwort (lokal), Accounts per DB-Script
- Keine Self-Service-Registrierung
- Drei Rollen: Lernende / Bereichsverantwortlicher / Admin
- Nicht authentifizierte Requests → Redirect auf Login-Seite
- **Post-MVP:** SSO via Azure AD / SAML 2.0 oder OIDC

Die Post-MVP-SSO-Anforderung ist der entscheidende Architektur-Treiber: die Auth-Strategie muss so gebaut werden, dass der Wechsel zu SSO kein Refactoring der Business-Logik, der RBAC-Middleware oder der geschützten Endpunkte erfordert.

6.2 Entscheidung

JWT-basierte Authentifizierung mit kurzlebigen Access-Token (15 min) und langlebigem Refresh-Token (7 Tage), beide als **HTTP-only Cookies**. Die gesamte Auth-Logik lebt in einer AuthMiddleware-Schicht hinter einem Interface. RBAC ist in einer separaten RBACMiddleware gekapselt.

6.2.1 Token-Strategie

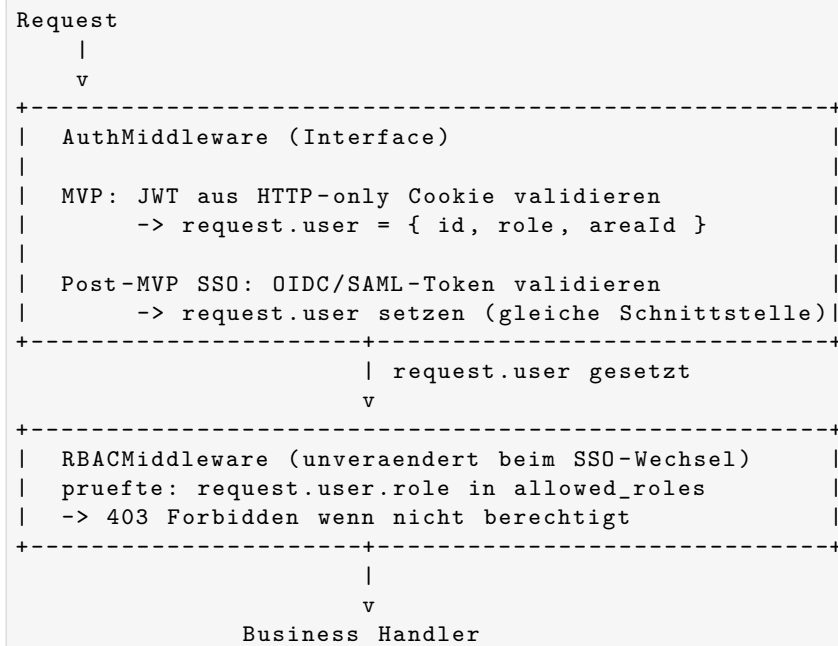
Token	Typ	Dauer	Transport	Inhalt
Access-Token	JWT (HS256)	15 min	HTTP-only Cookie	userId, role, areaId, exp
Refresh-Token	Opaque (UUID)	7 Tage	HTTP-only Cookie	in DB gespeichert

```

1 type AccessTokenPayload = {
2   sub:    string    // userId (UUID)
3   role:   Role      // 'learner' | 'area_manager' | 'admin'
4   areaId: string
5   exp:    number    // Unix Timestamp (now + 15 min)
6 }
7 type Role = 'learner' | 'area_manager' | 'admin'

```

6.2.2 Request-Flow



6.2.3 AuthMiddleware-Interface

```

1 interface AuthMiddleware {
2   authenticate(req: Request, res: Response, next: NextFunction):
     Promise<void>
3 }
4
5 class LocalJWTAuthMiddleware implements AuthMiddleware { ... } //
     MVP
6 class OIDCAuthMiddleware implements AuthMiddleware { ... } //
     Post-MVP
7
8 class MockAuthMiddleware implements AuthMiddleware {
9   constructor(private user: AuthUser) {}
10  async authenticate(req: any, res: any, next: NextFunction) {
11    req.user = this.user; next()
12  }
13 }
    
```

6.2.4 RBAC-Deklaration pro Route

```

1 router.post('/api/documents/upload',
2   rbac(['area_manager', 'admin']), uploadHandler)
3
4 router.get('/api/admin/config',
5   rbac(['admin']), configHandler)
6
7 function rbac(allowedRoles: Role[]): RequestHandler {
8   return (req, res, next) => {
9     if (!allowedRoles.includes((req as any).user?.role))
10       return res.status(403).json({ error: 'Forbidden' })
11     next()
12   }
13 }
    
```

6.2.5 Passwort-Hashing (Argon2id)

```
1 import argon2 from 'argon2'
2
3 const OPTIONS = { type: argon2.argon2id, memoryCost: 65536,
4                   timeCost: 3, parallelism: 4 }
5
6 async function hashPassword(plain: string): Promise<string> {
7   return argon2.hash(plain, OPTIONS)
8 }
9
10 async function verifyPassword(hash: string, plain: string):
11    Promise<boolean> {
12   return argon2.verify(hash, plain)
13 }
```

6.2.6 Datenbankschema

```
1 CREATE TABLE users (
2   id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
3   username    TEXT          NOT NULL UNIQUE,
4   password_hash TEXT        NOT NULL,
5   role        TEXT          NOT NULL CHECK (role IN
6     ('learner', 'area_manager', 'admin')),
7   area_id     TEXT          NOT NULL,
8   created_at  TIMESTAMPTZ   NOT NULL DEFAULT now(),
9   is_active   BOOLEAN       NOT NULL DEFAULT true
10 );
11
12 CREATE TABLE refresh_tokens (
13   id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
14   user_id     UUID          NOT NULL REFERENCES users(id) ON DELETE
15     CASCADE,
16   token_hash  TEXT          NOT NULL UNIQUE,      -- SHA-256 des Tokens
17   expires_at  TIMESTAMPTZ   NOT NULL,
18   created_at  TIMESTAMPTZ   NOT NULL DEFAULT now(),
19   revoked_at  TIMESTAMPTZ   -- NULL = aktiv
20 );
```

6.3 Begründung

Warum JWT + HTTP-only Cookie statt Server-Side Session? JWT-Access-Tokens sind zustandslos — Validierung ohne DB in < 1 ms. Der Refresh-Token in der DB ermöglicht trotzdem Server-seitige Invalidierung bei Logout oder Sicherheitsvorfällen.

Warum HTTP-only Cookie statt Authorization-Header? HTTP-only Cookies sind für JavaScript nicht lesbar — XSS-Angriffe können den Token nicht stehlen. Für ein internes Tool ohne öffentliche API ist dieser Trade-off optimal.

Warum Argon2id statt Bcrypt? Argon2id ist der aktuelle OWASP-Standard (2024) und resistent gegen GPU-basierte Brute-Force-Angriffe durch Memory-Hardness. Kosten: eine npm-Library.

Warum Refresh-Token-Rotation? Bei Rotation wird nach jedem Refresh der alte Token invalidiert und ein neuer ausgestellt. Ein gestohlener Refresh-Token wird beim nächsten legitimen Refresh erkannt (Collision → beide Sessions invalidieren).

Warum AuthMiddleware als Interface? Die Post-MVP-SSO-Anforderung ist kein Nice-to-Have. Ohne Interface zieht sich SAML/OIDC-Logik durch alle geschützten Endpunkte. Das Interface kapselt die Auth-Logik an genau einem Ort.

6.4 Betrachtete Alternativen

6.4.1 Alternative 1 – Auth.js (NextAuth.js)

Zurückgestellt Auth.js bietet native Next.js-Integration mit eingebautem OIDC/SAML-Support. Wenn ADR-007 Next.js wählt, sollte Auth.js als primäre Auth-Integration evaluiert werden — weniger Eigenimplementierung, dafür weniger Kontrolle über Token-Format. **Empfehlung:** Auth.js bevorzugen wenn Next.js als Framework gewählt.

6.4.2 Alternative 2 – Server-Side Session (express-session)

Abgelehnt DB-Lookup bei jedem Request. Horizontal schwerer skalierbar. SSO-Readiness erfordert dasselbe Interface-Konzept.

6.4.3 Alternative 3 – Keycloak (Self-Hosted IdP)

Abgelehnt Separater Server, erheblicher Setup- und Betriebsaufwand für 480h-MVP-Budget. Sinnvoll wenn das Unternehmen bereits einen zentralen IdP betreibt.

6.5 Konsequenzen

Positiv

- SSO-Wechsel (Post-MVP) erfordert ausschliesslich neue AuthMiddleware-Implementierung
- MockAuthMiddleware macht alle geschützten Endpunkte ohne Login-Flow testbar
- HTTP-only Cookies verhindern XSS-Token-Diebstahl
- Argon2id: state-of-the-art Passwortschutz
- Token-Rotation erkennt gestohlene Refresh-Tokens

Negativ / Risiken

- JWT-Invalidierung vor Ablauf nicht möglich ohne DB-Lookup. Mitigation: Refresh-Token revoked → nächste Rotation schlägt fehl
- HS256 (symmetrisch): JWT-Secret muss sicher im Secrets Manager liegen, nie in Code. Bei Multi-Instance wäre RS256 besser — für MVP-Single-Instance akzeptabel
- Argon2id-Hashing dauert $\approx 100\text{--}300$ ms (gewollt als Brute-Force-Schutz) — Login-Latenz entsprechend

6.6 Abhängigkeiten

Abhängigkeit	Typ	Hinweis
ADR-007 (Frontend-Framework)	Koordiniert	Wenn Next.js → Auth.js-Alternative bevorzugen
US-05	Direkt	Alle ACs von US-05 werden abgedeckt
QA-03 Security (M8, M9)	Validierung	RBAC server-side setzt diese Auth-Schicht voraus
ADR-003 (ConfigService)	Optional	JWT-Secret-Lebensdauer als Config-Parameter möglich

6.7 Offen / Nächste Schritte

- ☐ ADR-007 abschliessen: wenn Next.js → Auth.js-Alternative evaluieren
- ☐ JWT-Secret-Management definieren: Vault, Docker Secret oder ENV-Variable?
- ☐ DB-Script für erste Admin- und Testaccounts schreiben (Argon2id-Hashes)
- ☐ SSO-Provider-Typ (SAML 2.0 vs. OIDC) für Post-MVP-Planung klären

Frontend-Framework: Next.js mit TypeScript und App Router

Status	Proposed
Datum	2026-05-27
Autor	Christoph A. Amstutz, MD-PhD
Bezug	US-01, US-03, US-07-09, US-11, QA-05 (Maintainability)
Abhängigkeit	ADR-002 (Discriminated Union / TypeScript), ADR-006 (Auth-Strategie)

7.1 Kontext

LearnFlow benötigt vier funktional unterschiedliche UI-Bereiche:

Bereich	Hauptfunktionen	Besonderheiten
Chat-Interface (Lara)	Frage stellen, Antwort + Quellenlinks, Feedback, Folgefragen	LLM-Streaming
Lern-Interface (Lara)	Quiz absolvieren, Ergebnis	—
Dashboard (Stefan)	Dokumente verwalten, Quiz-Freigabe, Wissenslücken	Upload, Review
Admin-Seite	Schwellenwerte, Audit-Log	Formular-getrieben

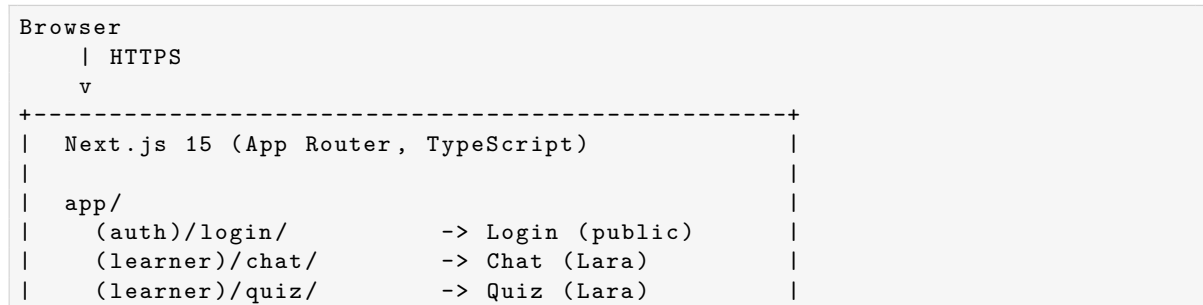
Aus ADR-002 ergibt sich eine technische Vorentscheidung: der Discriminated-Union-Exhaustiveness-Check ist nativ und vollständig typsicher nur in **TypeScript** umsetzbar. Das Framework muss TypeScript unterstützen.

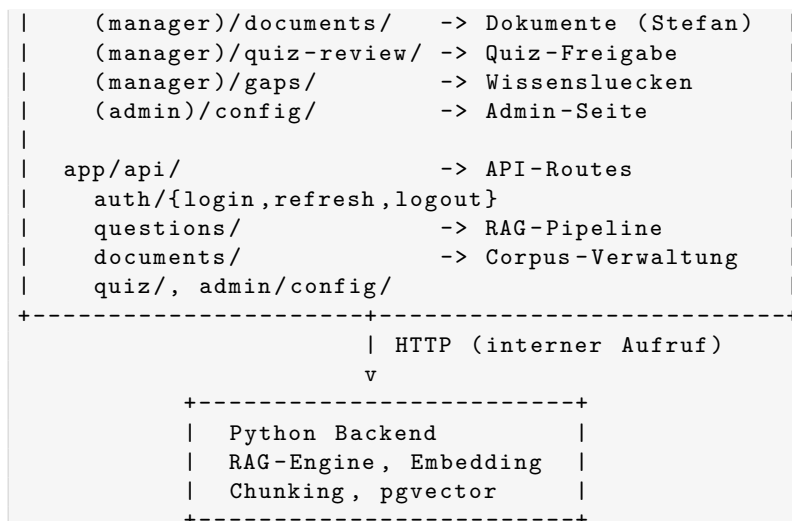
Scope: Desktop-Browser, kein Mobile als MVP-Anforderung.

7.2 Entscheidung

Next.js 15 (App Router) mit TypeScript als Full-Stack-Framework. Next.js übernimmt sowohl das Frontend (React-Komponenten) als auch das Backend (API-Routes) — kein separater API-Server für das MVP nötig.

7.2.1 Architektur-Übersicht





Die Python-Backend-Service-Grenze trennt das ML-Ökosystem (`sentence-transformers`, `pdfplumber`, `python-docx`) vom TypeScript-Frontend. Das ist die einzige Service-Grenze im MVP.

7.2.2 TypeScript als Pflicht (Verbindung zu ADR-002)

```
1 // Compiler-Fehler wenn ein RAGResult-Variant nicht behandelt wird
2 function renderResult(result: RAGResult): React.ReactNode {
3     switch (result.kind) {
4         case 'answer': return result.label === 'limited'
5                         ? <LimitedAnswerCard {...result} />
6                         : <AnswerCard {...result} />
7         case 'dont_know': return <DontKnowCard hint={result.hint} />
8         case 'no_source':
9         case 'suppressed': return <NoSourceCard />
10        case 'unavailable': return <ErrorCard message={result.message} />
11        case 'timeout': return <ErrorCard
12            message="Zeitüberschreitung." />
13        // Fehlender Case => TypeScript-Compilerfehler
14    }
15 }
```

7.2.3 LLM-Streaming im Chat-Interface (App Router)

```
1 // app/api/questions/route.ts
2 export async function POST(req: Request) {
3   const { question, history } = await req.json()
4
5   const stream = new ReadableStream({
6     async start(controller) {
7       const ragResult = await runRAGPipeline(question, history)
8       if (ragResult.kind !== 'answer') {
9         controller.enqueue(JSON.stringify(ragResult))
10        controller.close(); return
11      }
12      for await (const chunk of llmAdapter.stream(ragResult.prompt))
13        controller.enqueue(chunk)
14      controller.close()
15    }
16  })
17 }
```

```

17   return new Response(stream, { headers: { 'Content-Type':
18     'text/event-stream' }})
  }

```

7.2.4 Auth.js-Integration für Post-MVP SSO

```

1  // auth.ts (wenn Auth.js gewaehlt -- Alternative 1 aus ADR-006)
2  import NextAuth from 'next-auth'
3  import Credentials from 'next-auth/providers/credentials'
4
5  export const { handlers, auth, signIn, signOut } = NextAuth({
6    providers: [
7      Credentials({
8        async authorize({ username, password }) {
9          return await verifyUser(username as string, password as
10             string) ?? null
11        }
12      })
13      // Post-MVP: SAML/OIDC-Provider hier ergaenzen -- kein Code
14      // unterhalb aendert sich
15    ],
16    callbacks: {
17      jwt({ token, user }) {
18        if (user) { token.role = user.role; token.areaId =
19          user.areaId }
20        return token
21      }
22    }
23  })

```

7.2.5 Verzeichnisstruktur

```

learnflow/
  app/
    (auth)/login/page.tsx
    (learner)/chat/page.tsx
    (learner)/quiz/page.tsx
    (manager)/documents/
    (manager)/quiz-review/
    (admin)/config/
    api/{auth,questions,documents,quiz,admin}/
  components/
    chat/{AnswerCard,LimitedAnswerCard,DontKnowCard,NoSourceCard,ErrorCard}.tsx
    shared/
  lib/
    rag/           -- RAGResult-Typen, Pipeline-Client
    auth/          -- AuthMiddleware-Interface + Implementierung
    config/        -- ConfigService-Client
    types/rag.ts   -- RAGResult Discriminated Union (ADR-002)
    middleware.ts  -- Auth-Schutz aller Routen

```

7.3 Begründung

Warum Next.js statt React SPA + Express? Zwei separate Services zu deployen und zu betreiben erhöht den Aufwand für ein 1-FTE-Maintenance-Team erheblich. Next.js (ein Deployment, kein CORS, ein Dev-Server) ist die ökonomischere Wahl. API-Routes sind vollwertige Node.js-Handler — kein Funktionsverlust.

Warum App Router statt Pages Router? App Router bietet native Streaming-Unterstützung via React 18 Suspense und `ReadableStream`. Das ist für die LLM-Streaming-Anforderung ($p_{95} \leq 10\text{s}$, US-09) der entscheidende Vorteil.

Warum nicht SvelteKit? Svelte ist schneller und leichtgewichtiger. Der Hauptgrund dagegen: der Discriminated-Union-Exhaustiveness-Check (ADR-002) ist in Svelte-Templates nicht compiler-erzwingbar. TypeScript-Sicherheit ist hier keine optionale Verbesserung, sondern eine Kernanforderung.

Warum Python für Backend-Services? Das ML-Ökosystem für Embedding (`sentence-transformers`), PDF-Parsing (`pdfplumber`) und zukünftige Scoring-Modelle ist in Python deutlich reifer. Eine einzelne Service-Grenze hält diese Stärke nutzbar ohne die Frontend-Entwicklung auf Python zu zwingen.

7.4 Betrachtete Alternativen

7.4.1 Alternative 1 – SvelteKit + TypeScript

Abgelehnt Exhaustiveness-Check für Discriminated Unions in Svelte-Templates nicht erzwingbar. Kleineres Ökosystem für Auth (Post-MVP SSO).

7.4.2 Alternative 2 – React SPA (Vite) + Express.js

Abgelehnt Zwei separate Services. CORS-Konfiguration. Kein natives Streaming ohne manuelle SSE-Implementierung. Kein Mehrwert gegenüber Next.js.

7.4.3 Alternative 3 – Vue 3 + Nuxt

Abgelehnt Kein technischer Vorteil gegenüber Next.js. Auth.js-Integration für Nuxt schwächer als für Next.js.

7.5 Konsequenzen

Positiv

- Ein einziges Deployment für Frontend und BFF-API
- TypeScript end-to-end: Discriminated Unions aus ADR-002 voll typsicher
- Natives Streaming für LLM-Antworten (App Router + React 18)
- Auth.js deckt Post-MVP SSO ohne Refactoring
- Route-Groups (`(admin)`, `(learner)`) isolieren RBAC

Negativ / Risiken

- App Router jünger als Pages Router — mehr API-Instabilität in edge cases
- Python-Backend als zweiter Service: eine Service-Grenze; Mitigation: Docker Compose für lokale Entwicklung
- Server Components vs. Client Components: erhöhter Erklärungsaufwand beim Onboarding

7.6 Abhängigkeiten

7.7 Offen / Nächste Schritte

- ☐ ADR-006 aktualisieren: Auth.js als primäre Auth-Integration bestätigen

Abhängigkeit	Typ	Hinweis
ADR-002 (Discriminated Union)	Voraussetzung	TypeScript ist Pflicht — Next.js erfüllt das
ADR-006 (Auth-Strategie)	Koordiniert	Auth.js als bevorzugte MVP-Auth
ADR-005 (RAG-Stack / Python)	Nachgelagert	Next.js API-Routes kommunizieren via HTTP mit Python-Backend
US-09 (Folgefragen)	Feature	Konversationshistorie (max. 3 Paare) im Client-State

- ☐ Python-Backend-Interface definieren: OpenAPI-Spec oder TypeScript-Types für den HTTP-Kontrakt
- ☐ Docker-Compose-Setup: Next.js + Python-Backend + PostgreSQL (mit pgvector)
- ☐ Styling-Strategie entscheiden: shadcn/ui oder Tailwind CSS direkt
- ☐ Streaming-Implementierung prototypisch testen — p_{95} -Messung mit Mock-LLM und realem Netzwerk